

Sampling nonlinear posteriors in the steady state sensor model

a SMARTWINE presentation



David A. Birkenmayer

AG Optimierung, RPTU Kaiserslautern

Supervisor: Professor Sven O. Krumke

Fixed values

- *wdn* as digraph $G = (V, E)$, where
 $V = \mathcal{J} \cup \{v_0\}$ with v_0 reservoir node
- *pipe resistances* $r_e > 0$ for $e \in E$

Fixed values

- *wdn* as digraph $G = (V, E)$, where
 $V = \mathcal{J} \cup \{v_0\}$ with v_0 reservoir node
- *pipe resistances* $r_e > 0$ for $e \in E$

Variables:

- *demand* $d_v > 0$ for $v \in \mathcal{J}$
- *flow* q_e for $e \in E$ with $q_{v,u} := -q_{u,v}$
- *pressure head* h_v for $v \in \mathcal{J}$

Fixed values

- *wdn* as digraph $G = (V, E)$, where $V = \mathcal{J} \cup \{v_0\}$ with v_0 reservoir node
- *pipe resistances* $r_e > 0$ for $e \in E$
- *reservoir pressure* $H_0 \in \mathbb{R}$

Variables:

- *demand* $d_v > 0$ for $v \in \mathcal{J}$
- *flow* q_e for $e \in E$ with $q_{v,u} := -q_{u,v}$
- *pressure head* h_v for $v \in \mathcal{J}$
- *total demand* $D > 0$

Fixed values

- *wdn* as digraph $G = (V, E)$, where $V = \mathcal{J} \cup \{v_0\}$ with v_0 reservoir node
- *pipe resistances* $r_e > 0$ for $e \in E$
- *reservoir pressure* $H_0 \in \mathbb{R}$

Variables:

- *demand* $d_v > 0$ for $v \in \mathcal{J}$
- *flow* q_e for $e \in E$ with $q_{v,u} := -q_{u,v}$
- *pressure head* h_v for $v \in \mathcal{J}$
- *total demand* $D > 0$

Physical equations:

$$\mathbb{1}^\top d = D \quad (\text{total demand})$$

$$Bq = -d \quad (\text{mass})$$

$$B^\top h = \phi(q) - B_0^\top H_0 \quad (\text{energy})$$

- $B \in \mathbb{R}^{\mathcal{J} \times E}$ is the v_0 -reduced incidence matrix of G
- B_0 is eliminated v_0 -row of the incidence matrix
- $\phi(x)_e := r_e \cdot q_e \cdot |q_e|^{\kappa-1}$ the nonlinear engery function

Fixed values

- *wdn* as digraph $G = (V, E)$, where $V = \mathcal{J} \cup \{v_0\}$ with v_0 reservoir node
- *pipe resistances* $r_e > 0$ for $e \in E$
- *reservoir pressure* $H_0 \in \mathbb{R}$

Physical equations:

$$\mathbb{1}^\top d = D \quad (\text{total demand})$$

$$Bq = -d \quad (\text{mass})$$

$$B^\top h = \phi(q) - B_0^\top H_0 \quad (\text{energy})$$

- $B \in \mathbb{R}^{\mathcal{J} \times E}$ is the v_0 -reduced incidence matrix of G
- B_0 is eliminated v_0 -row of the incidence matrix
- $\phi(x)_e := r_e \cdot q_e \cdot |q_e|^{\kappa-1}$ the nonlinear engery function

Variables:

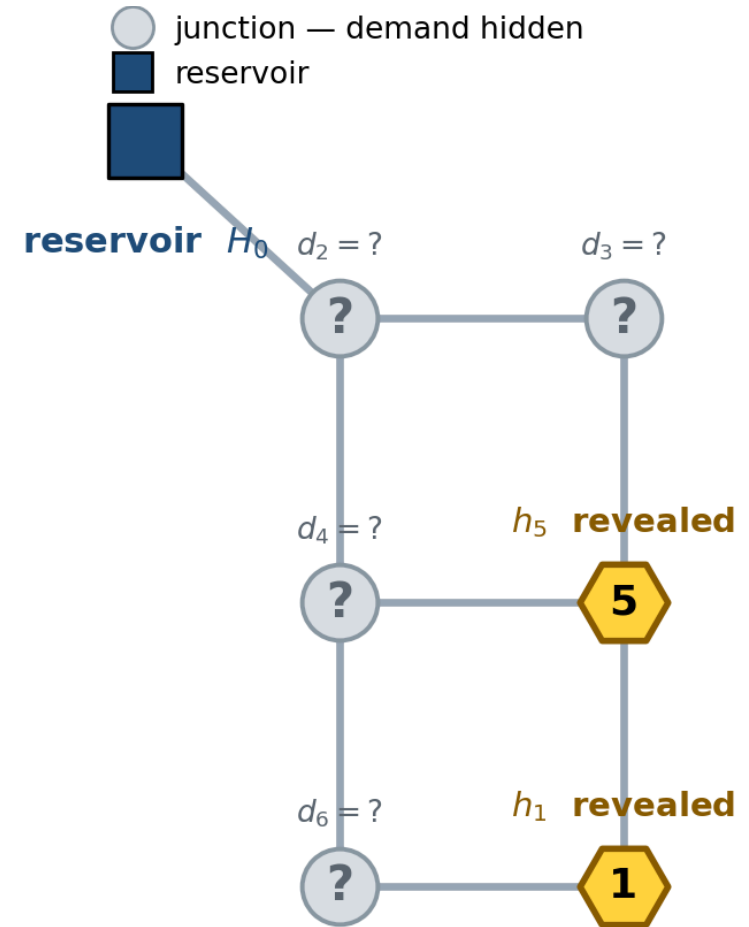
- *demand* $d_v > 0$ for $v \in \mathcal{J}$
- *flow* q_e for $e \in E$ with $q_{v,u} := -q_{u,v}$
- *pressure head* h_v for $v \in \mathcal{J}$
- *total demand* $D > 0$

What we know:

- Given demands d and D , pressures h are unique. Calculation is well-conditioned.
- Given pressures h , demands d and D are unique. Calculation is ill-conditioned around low flows.

Hydraulic states:

- a *hydraulic state* is a tuple $s = (d^s, q^s, h^s, D^s)$ fulfilling the physical equations
- S is the set of all hydraulic states
- we assume to have a *prior distribution* P on S

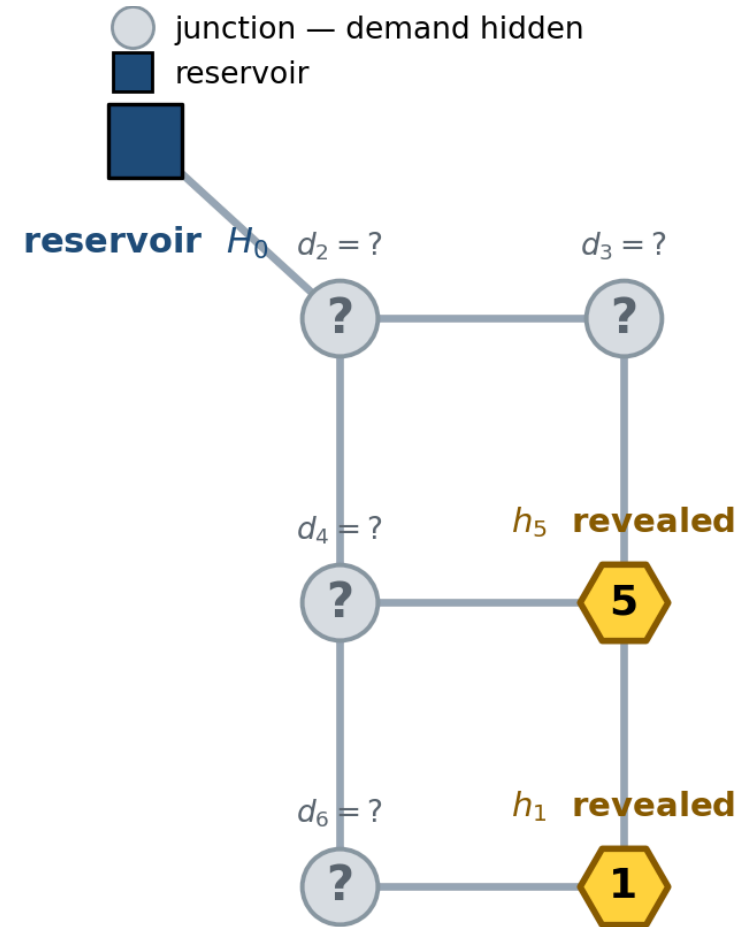


Hydraulic states:

- a *hydraulic state* is a tuple $s = (d^s, q^s, h^s, D^s)$ fulfilling the physical equations
- S is the set of all hydraulic states
- we assume to have a *prior distribution* P on S

Measurements

- a *site configuration* is a subset $\mathcal{Y} \subseteq \mathcal{J}$, where pressure measuring sensors are placed
- an *observation* $z = (D^z, h_y^z)$ is a tuple of the observed total demand and the observed pressures at the sites
- Z is the set of all observations.

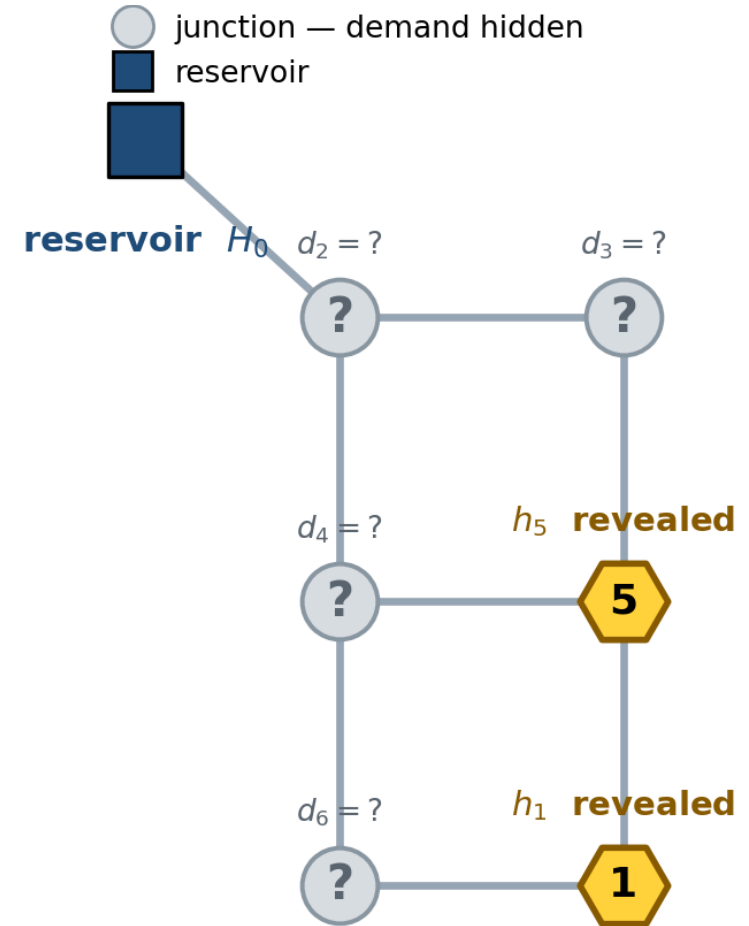


Hydraulic states:

- a *hydraulic state* is a tuple $s = (d^s, q^s, h^s, D^s)$ fulfilling the physical equations
- S is the set of all hydraulic states
- we assume to have a *prior distribution* P on S

Measurements

- a *site configuration* is a subset $\mathcal{Y} \subseteq \mathcal{J}$, where pressure measuring sensors are placed
- an *observation* $z = (D^z, h_y^z)$ is a tuple of the observed total demand and the observed pressures at the sites
- Z is the set of all observations.
- assume a *sensor likelihood* function $P(z \mid s)$



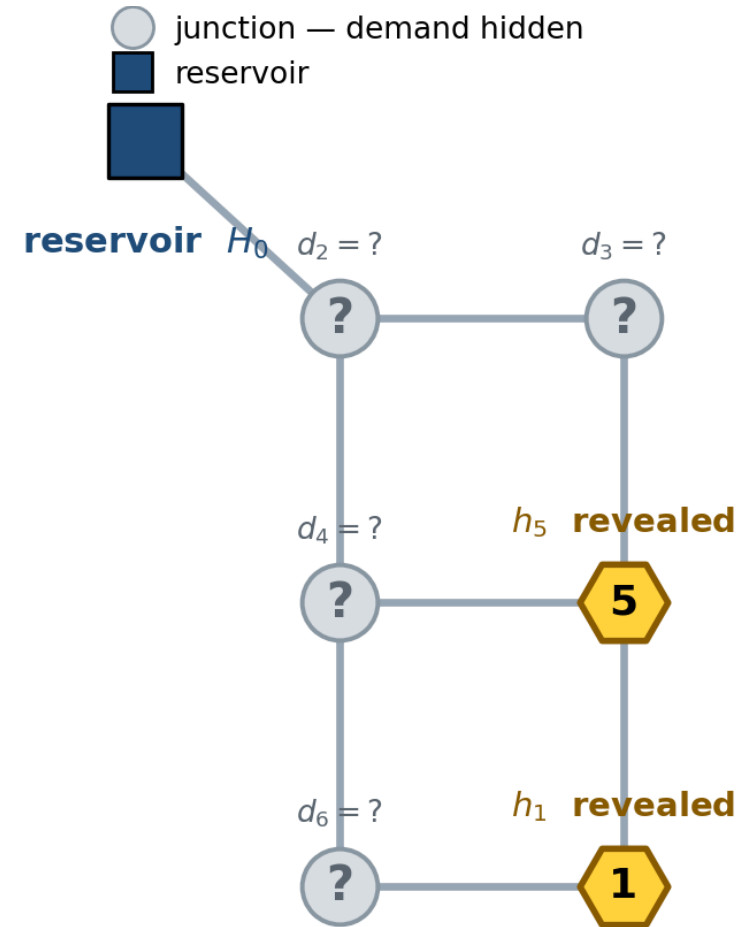
Hydraulic states:

- a *hydraulic state* is a tuple $s = (d^s, q^s, h^s, D^s)$ fulfilling the physical equations
- S is the set of all hydraulic states
- we assume to have a *prior distribution* P on S

Measurements

- a *site configuration* is a subset $\mathcal{Y} \subseteq \mathcal{J}$, where pressure measuring sensors are placed
- an *observation* $z = (D^z, h_y^z)$ is a tuple of the observed total demand and the observed pressures at the sites
- Z is the set of all observations.
- assume a *sensor likelihood* function $P(z \mid s)$

→ Want to find a good predictor $F: Z \rightarrow S$ (e.g. GNN)



Hydraulic states:

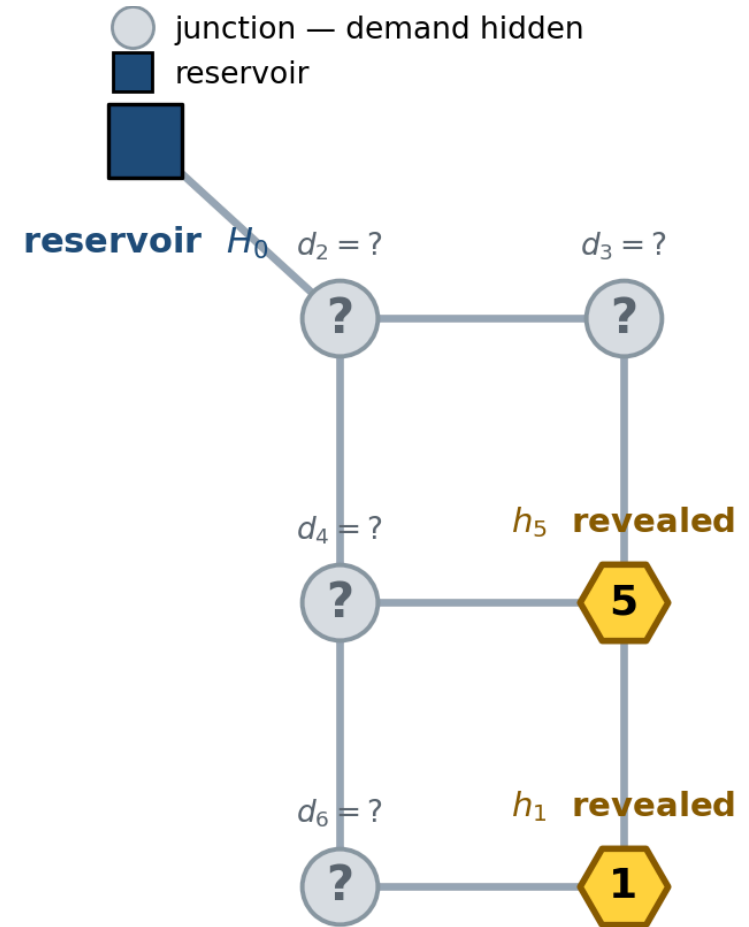
- a *hydraulic state* is a tuple $s = (d^s, q^s, h^s, D^s)$ fulfilling the physical equations
- S is the set of all hydraulic states
- we assume to have a *prior distribution* P on S

Measurements

- a *site configuration* is a subset $\mathcal{Y} \subseteq \mathcal{J}$, where pressure measuring sensors are placed
- an *observation* $z = (D^z, h_y^z)$ is a tuple of the observed total demand and the observed pressures at the sites
- Z is the set of all observations.
- assume a *sensor likelihood* function $P(z \mid s)$

→ Want to find a good predictor $F: Z \rightarrow S$ (e.g. GNN)

→ But how do we determine what “good” is?



Assumptions:

- *prior distribution* P on the state space $\mathcal{S}$
- *sensor likelihood* $P(z \mid s)$
- *measurement sites* $\mathcal{Y} \subseteq \mathcal{J}$ (hexagons)
- *observation* $z = (D^z, h_y^z)$

Assumptions:

- *prior distribution* P on the state space S
- *sensor likelihood* $P(z \mid s)$
- *measurement sites* $\mathcal{Y} \subseteq \mathcal{J}$ (hexagons)
- *observation* $z = (D^z, h_y^z)$

What we want to do:

→ sample *posterior distribution*

$$P_z(s) := P(s \mid z) \propto \underbrace{P(s)}_{\text{prior}} \cdot \underbrace{P(z \mid s)}_{\text{sensor likelihood}}$$

Assumptions:

- *prior distribution* P on the state space S
- *sensor likelihood* $P(z \mid s)$
- *measurement sites* $\mathcal{Y} \subseteq \mathcal{J}$ (hexagons)
- *observation* $z = (D^z, h_y^z)$

What we want to do:

→ sample *posterior distribution*

$$P_z(s) := P(s \mid z) \propto \underbrace{P(s)}_{\text{prior}} \cdot \underbrace{P(z \mid s)}_{\text{sensor likelihood}}$$

Assumptions:

- *prior distribution* P on the state space S
- *sensor likelihood* $P(z \mid s)$
- *measurement sites* $\mathcal{Y} \subseteq \mathcal{J}$ (hexagons)
- *observation* $z = (D^z, h_y^z)$

What we want to do:

→ sample *posterior distribution*

$$P_z(s) := P(s \mid z) \propto \underbrace{P(s)}_{\text{prior}} \cdot \underbrace{P(z \mid s)}_{\text{sensor likelihood}}$$

Why do we want this:

- Can measure how good a prediction $F(z)$ is, by comparing it to the sampled results
- Shows how confident we can be in the prediction in certain areas
- Averaging the samples yields a predictor itself!

Assumptions:

- *prior distribution* P on the state space S
- *sensor likelihood* $P(z \mid s)$
- *measurement sites* $\mathcal{Y} \subseteq \mathcal{J}$ (hexagons)
- *observation* $z = (D^z, h_y^z)$

What we want to do:

→ sample *posterior distribution*

$$P_z(s) := P(s \mid z) \propto \underbrace{P(s)}_{\text{prior}} \cdot \underbrace{P(z \mid s)}_{\text{sensor likelihood}}$$

Why do we want this:

- Can measure how good a prediction $F(z)$ is, by comparing it to the sampled results
- Shows how confident we can be in the prediction in certain areas
- Averaging the samples yields a predictor itself!

Assumptions:

- *prior distribution* P on the state space S
- *sensor likelihood* $P(z \mid s)$
- *measurement sites* $\mathcal{Y} \subseteq \mathcal{J}$ (hexagons)
- *observation* $z = (D^z, h_y^z)$

What we want to do:

→ sample *posterior distribution*

$$P_z(s) := P(s \mid z) \propto \underbrace{P(s)}_{\text{prior}} \cdot \underbrace{P(z \mid s)}_{\text{sensor likelihood}}$$

Why do we want this:

- Can measure how good a prediction $F(z)$ is, by comparing it to the sampled results
- Shows how confident we can be in the prediction in certain areas
- Averaging the samples yields a predictor itself!

Bayes Rule: the measurement *reweights* the prior $P_z(s) \propto \underbrace{P(z \mid s)}_{\text{sensor likelihood}} \cdot \underbrace{P(s)}_{\text{prior}}$

Bayes Rule: the measurement *reweights* the prior $P_z(s) \propto \underbrace{P(z \mid s)}_{\text{sensor likelihood}} \cdot \underbrace{P(s)}_{\text{prior}}$

Prior defined on the demands

- Dirichlet: Given d^0 and $\Delta > 0$, uniform distribution on simplex $\{d \geq d^0 : \mathbb{1}^\top d \leq \mathbb{1}^\top d^0 + \Delta\}$
- Gaussian: Normal distribution around d^0 with a std

Bayes Rule: the measurement *reweights* the prior $P_z(s) \propto \underbrace{P(z \mid s)}_{\text{sensor likelihood}} \cdot \underbrace{P(s)}_{\text{prior}}$

Prior defined on the demands

- Dirichlet: Given d^0 and $\Delta > 0$, uniform distribution on simplex $\{d \geq d^0 : \mathbb{1}^\top d \leq \mathbb{1}^\top d^0 + \Delta\}$
- Gaussian: Normal distribution around d^0 with a std

Sensor likelihood function K_ε :

- $P(z \mid s)$: encodes what we believe about *sensor noise*
- We split:
$$P(z \mid s) \propto \underbrace{\delta(D^z - D^s)}_{\text{total demand: exact}} \cdot \underbrace{K_\varepsilon(h_y^s - h_y^z)}_{\text{pressures: soft}}$$
- Three choices for sensor likelihood

Dirac $\delta(h_y^s - h_y^z)$ no sensor error – hard constraint

Uniform $\mathbb{1}[\|h_y^s - h_y^z\| < \varepsilon]$ sensor error bounded by ε

Gaussian $\exp(-\frac{1}{2\varepsilon^2} \|h_y^s - h_y^z\|^2)$ gaussian sensor error with std ε

Bayes Rule: the measurement *reweights* the prior $P_z(s) \propto \underbrace{P(z \mid s)}_{\text{sensor likelihood}} \cdot \underbrace{P(s)}_{\text{prior}}$

Prior defined on the demands

- Dirichlet: Given d^0 and $\Delta > 0$, uniform distribution on simplex $\{d \geq d^0 : \mathbb{1}^\top d \leq \mathbb{1}^\top d^0 + \Delta\}$
- Gaussian: Normal distribution around d^0 with a std

Sensor likelihood function K_ε :

- $P(z \mid s)$: encodes what we believe about *sensor noise*
- We split:
$$P(z \mid s) \propto \underbrace{\delta(D^z - D^s)}_{\text{total demand: exact}} \cdot \underbrace{K_\varepsilon(h_y^s - h_y^z)}_{\text{pressures: soft}}$$
- Three choices for sensor likelihood

Dirac $\delta(h_y^s - h_y^z)$ no sensor error – hard constraint

Uniform $\mathbb{1}[\|h_y^s - h_y^z\| < \varepsilon]$ sensor error bounded by ε

Gaussian $\exp(-\frac{1}{2\varepsilon^2} \|h_y^s - h_y^z\|^2)$ gaussian sensor error with std ε

- $\varepsilon > 0$ softens; both Uniform and Gaussian $\rightarrow$ Dirac as $\varepsilon \rightarrow 0$
- Question: Which combination of Prior and Sensor likelihood is best?

Maximum A-Posteriori Estimation (MAP) [1]



- Steady-state-model is *linearized around a fixed reference point* $\hat{s}$
- Demands are gaussian-distributed around $d^{\hat{s}}$

Maximum A-Posteriori Estimation (MAP) [1]



- Steady-state-model is *linearized around a fixed reference point* $\hat{s}$
- Demands are gaussian-distributed around $d^{\hat{s}}$
- Use gaussian sensor-likelihood function:

$$P(z \mid s) \propto \exp\left(-\frac{1}{2\epsilon^2}\|h_y^s - h_y^z\|^2\right)$$

Maximum A-Posteriori Estimation (MAP) [1]



- Steady-state-model is *linearized around a fixed reference point* $\hat{s}$
- Demands are gaussian-distributed around $d^{\hat{s}}$
- Use gaussian sensor-likelihood function:

$$P(z \mid s) \propto \exp\left(-\frac{1}{2\epsilon^2}\|h_y^s - h_y^z\|^2\right)$$

- Idea: By linearity the mean is equal to the *mode* (most likely state):

- Steady-state-model is *linearized around a fixed reference point* $\hat{s}$
- Demands are gaussian-distributed around $d^{\hat{s}}$
- Use gaussian sensor-likelihood function:

$$P(z \mid s) \propto \exp\left(-\frac{1}{2\epsilon^2}\|h_y^s - h_y^z\|^2\right)$$

- Idea: By linearity the mean is equal to the *mode* (most likely state):

$$\begin{aligned}\arg \max_s P(s \mid z) &= \arg \max_s \log(P(z \mid s)) + \log(P(s)) \\ &= \arg \min_s \frac{1}{2\epsilon^2}\|h_y^s - h_y^z\|^2 + \frac{1}{2}\|d^s - d^{\hat{s}}\|^2\end{aligned}$$

- This yields a smallest squares problem which can be solved by Newton's method
- posterior distribution is approximated with *laplace-approximation* (gives covariance)

- Steady-state-model is *linearized around a fixed reference point* $\hat{s}$
- Demands are gaussian-distributed around $d^{\hat{s}}$
- Use gaussian sensor-likelihood function:

$$P(z \mid s) \propto \exp\left(-\frac{1}{2\varepsilon^2}\|h_y^s - h_y^z\|^2\right)$$

- Idea: By linearity the mean is equal to the *mode* (most likely state):

$$\begin{aligned}\arg \max_s P(s \mid z) &= \arg \max_s \log(P(z \mid s)) + \log(P(s)) \\ &= \arg \min_s \frac{1}{2\varepsilon^2}\|h_y^s - h_y^z\|^2 + \frac{1}{2}\|d^s - d^{\hat{s}}\|^2\end{aligned}$$

- This yields a smallest squares problem which can be solved by Newton's method
 - posterior distribution is approximated with *laplace-approximation* (gives covariance)
- | | | | |
|---|---|---|---|
| + | fast and reliable solution | - | only works with Gaussian prior |
| - | requires fixed reference and linearizes | - | posterior distribution is only approximated |

- *Algorithm:* Sample $s \sim P$, and accept it with probability $P(z \mid s)$
- for uniform likelihood, would be accepted if $\|h_y^s - h_y^z\| < \varepsilon$
- for gaussian likelihood, would be accepted with probability $\exp(-\frac{1}{2\varepsilon^2} \|h_y^s - h_y^z\|^2)$
- dirac not applicable, since acceptance rate would be zero.

- *Algorithm:* Sample $s \sim P$, and accept it with probability $P(z \mid s)$
 - for uniform likelihood, would be accepted if $\|h_y^s - h_y^z\| < \varepsilon$
 - for gaussian likelihood, would be accepted with probability $\exp(-\frac{1}{2\varepsilon^2} \|h_y^s - h_y^z\|^2)$
 - dirac not applicable, since acceptance rate would be zero.
- + Works with any prior, does not require linearization
- Acceptance rate scales with $(\frac{1}{\varepsilon})^{|\mathcal{Y}|}$ (curse of dimensionality)

- *Algorithm:* Sample $s \sim P$, and accept it with probability $P(z \mid s)$
- for uniform likelihood, would be accepted if $\|h_y^s - h_y^z\| < \varepsilon$
- for gaussian likelihood, would be accepted with probability $\exp(-\frac{1}{2\varepsilon^2} \|h_y^s - h_y^z\|^2)$
- dirac not applicable, since acceptance rate would be zero.
- + Works with any prior, does not require linearization
- Acceptance rate scales with $(\frac{1}{\varepsilon})^{|y|}$ (curse of dimensionality)
- Essentially uses uniform sensor-likelihood

$$P_{\text{unf}}(z \mid s) \propto \begin{cases} 1 & \text{if } \|h_y^s - h_y^z\| < \varepsilon \\ 0 & \text{otherwise} \end{cases}$$

Monte Carlo Markov Chain (MCMC)



- similar to ABC method, but uses dependent generated samples $(s_0, s_1, \dots)$ instead of independent samples

- similar to ABC method, but uses dependent generated samples $(s_0, s_1, \dots)$ instead of independent samples

Algorithm:

- Assume we have already chosen $s_1, \dots, s_n$
- Propose candidate s_{n+1} by adding noise to s_n
- If $\mathcal{L}(s_{n+1}) \geq \mathcal{L}(s_n)$, accept the candidate
- If not, accept with probability $\frac{\mathcal{L}(s_{n+1})}{\mathcal{L}(s_n)} < 1$

- similar to ABC method, but uses dependent generated samples $(s_0, s_1, \dots)$ instead of independent samples

Algorithm:

- Assume we have already chosen $s_1, \dots, s_n$
- Propose candidate s_{n+1} by adding noise to s_n
- If $\mathcal{L}(s_{n+1}) \geq \mathcal{L}(s_n)$, accept the candidate
- If not, accept with probability $\frac{\mathcal{L}(s_{n+1})}{\mathcal{L}(s_n)} < 1$
- produces correct posterior if noise is symmetric (e.g. gaussian)
- discard around first half of samples (burn-in period)

- similar to ABC method, but uses dependent generated $(s_0, s_1, \dots)$ instead of independent samples

Algorithm:

- Assume we have already chosen $s_1, \dots, s_n$
- Propose candidate s_{n+1} by adding noise to s_n
- If $\mathcal{L}(s_{n+1}) \geq \mathcal{L}(s_n)$, accept the candidate
- If not, accept with probability $\frac{\mathcal{L}(s_{n+1})}{\mathcal{L}(s_n)} < 1$
- produces correct posterior if noise is symmetric (e.g. gaussian)
- discard around first half of samples (burn-in period)
- + Works with any prior, does not require linearization
- + stays in high-likelihood-regions
- samples are *autocorrelated*, need more than usual
- Algorithm only works if it actually *mixes*
(chain eventually independent of starting point)

Exact Pressure Space approach

- Sample pressure vectors h_v for $v \in V \setminus \mathcal{Y}$, (we know $h_y = h_y^z$)
- Calculate corresponding d .
- Enforce $\mathbb{1}^\top d = D$ by changing $h_{v'}$ accordingly, leads to hydraulic state s
- calculate likelihood of state occurring via Jacobian determinant

Exact Pressure Space approach

- Sample pressure vectors h_v for $v \in V \setminus \mathcal{Y}$, (we know $h_y = h_y^z$)
- Calculate corresponding d .
- Enforce $\mathbb{1}^\top d = D$ by changing $h_{v'}$ accordingly, leads to hydraulic state s
- calculate likelihood of state occurring via Jacobain determinant

Exact Demand Space approach

- Sample demand vectors d (all of them)
- Calculate corresponding h .
- Enforce $\mathbb{1}^\top d = D$ and $h_y = h_y^z$ by changing d_y and $d_{v'}$ accordingly, leads to hydraulic state s
- calculate likelihood of state occurring via Jacobain determinant

Exact Pressure Space approach

- Sample pressure vectors h_v for $v \in V \setminus \mathcal{Y}$, (we know $h_y = h_y^z$)
- Calculate corresponding d .
- Enforce $\mathbb{1}^\top d = D$ by changing $h_{v'}$ accordingly, leads to hydraulic state s
- calculate likelihood of state occurring via Jacobain determinant

Exact Demand Space approach

- Sample demand vectors d (all of them)
- Calculate corresponding h .
- Enforce $\mathbb{1}^\top d = D$ and $h_y = h_y^z$ by changing d_y and $d_{v'}$ accordingly, leads to hydraulic state s
- calculate likelihood of state occurring via Jacobain determinant

Inexact Demand Space approach:

- Instead of adjusting d w.r.t $h_y = h_y^z$, use gaussian sensor-likelihood function

Correctness: every sampler cross-validated against an independent *ABC oracle*
(Alperovits, agreement in posterior- σ):

sampler	gap to oracle
<i>MAP</i> (Gaussian, MAP)	0.04σ
<i>MCMC</i> pressure-space (exact)	0.08σ
<i>MCMC</i> demand-space (gaussian sensor)	0.07σ
<i>MCMC</i> demand-space (exact)	0.01σ
<i>MCMC</i> Gaussian prior	0.07σ

→ MAP $\approx$ MCMC-mean when near-Gaussian; MCMC recovers the *non-Gaussian* shape.

Bibliography

- [1] Y. Shao, S. Chu, T. Zhang, Y. J. Yang, and T. Yu, “Real-Time Water Distribution System Hydraulic Modeling Using Prior Demand Information by Formal Bayesian Approach,” *Journal of Water Resources Planning and Management*, vol. 145, no. 12, p. 10.1061/(asce)wr.1943–5452.0001137, 2019, doi: 10.1061/(asce)wr.1943-5452.0001137.
- [2] M. Sunnåker, A. G. Busetto, E. Numminen, J. Corander, M. Foll, and C. Dessimoz, “Approximate Bayesian Computation,” *PLOS Computational Biology*, vol. 9, no. 1, p. e1002803, 2013, doi: 10.1371/journal.pcbi.1002803.

Thank you for your attention!

Shoutout to Fabian vdW for the Slide design